Algorithme augmenté par l'apprentissage pour la couverture de points sur la ligne par un drone

Hlib Kharytonov Christoph Dürr Bruno Escoffier

Juillet 2026

Abstract

On considère un problème de couverture de points par un drone d'observation. Les points sont tirés selon une loi normale, et le drone apprend les paramètres de la loi au fur et à mesure des tirages. Le but est de faire le moins de déplacements possibles pour couvrir tous les points, qui apparaissent au fur et à mesure. Différents algorithmes pour ce problème seront comparés expérimentalement.

1 Le problème

Le problème consiste à déplacer un drone dans le plan, pour couvrir des points qui apparaissent au fur et à mesure au sol, voir Fig. 1. Un drone positionné en (x, y), avec $y \geq 0$, couvre tous les points dans l'intervalle $[x - d, x + d]$ pour $d = y \cdot \sin(\alpha)/\cos(\alpha)$. Ici $\alpha = \frac{\pi}{4}$ est une constante fixée du problème. La position initiale du problème est $(0, 0)$. La séquence des points à couvrir $p_1, p_2, ..., p_n \in \mathbb{R}$ est révélé au fur et à mesure. Le coût d'un algorithme est la distance totale parcourue. Cette distance est normalisée par le coût optimal. Ce dernier vaut la distance entre $(0, 0)$ et l'unique point (x, y) dont la couverture est exactement $[\min p_i, \max p_i]$. Le rapport entre ces deux coûts est appelé le *rapport de compétitivité*.

2 Travaux reliés

Ce problème fut introduit par Debrev et al dans [1] et [2]. Les auteurs y analysent le ratio de compétitivité de trois algorithmes:

- Greedy, qui se déplace en angle α vers le point le plus proche qui couvre la nouvelle requête (en plus des requêtes précédentes).
- Straight-Up, qui se déplace seulement de manière verticale, donc en angle 0.
- β-Hedge, qui est un compromis entre les deux derniers algorithmes. Il se déplace en angle β jusqu'à couvrir la nouvelle requête, où β est un angle choisi pour optimiser le ratio de compétitivité.

Les résultats obtenu dans [2] sont les suivants.

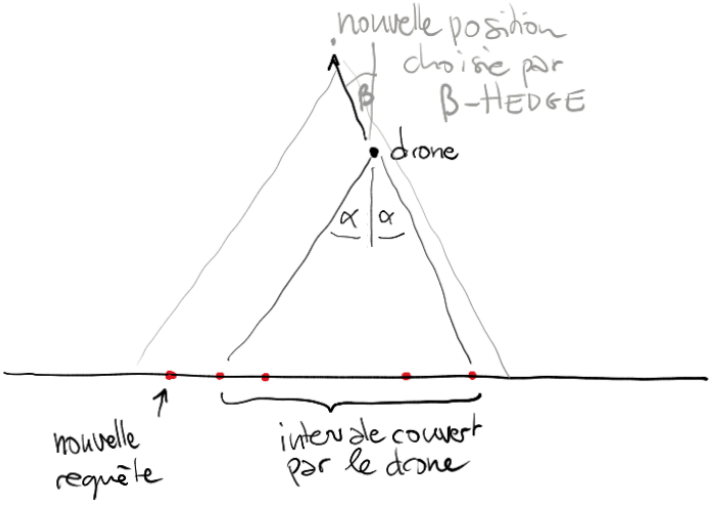


Fig. 1: Illustration du problème

- Pour $\frac{\pi}{3} \leq \alpha < \frac{\pi}{2}$, β-HEDGE se comporte comme STRAIGHT-UP.
- Pour $0 < \alpha \leq \frac{\pi}{6}$, β-HEDGE se comporte comme GREEDY.
- Pour $\frac{\pi}{6} < \alpha < \frac{\pi}{3}$, β-HEDGE est le meilleur des trois algorithmes. Son ratio est le pire pour $\alpha = \frac{\pi}{4}$ et y vaut 1.25, là où les deux autres algorithmes ont le ratio $\sqrt{2}$.

Pour $\frac{\pi}{6} < \alpha < \frac{\pi}{3}$, β-HEDGE est le meilleur des trois algorithmes[1]. Son ratio est le pire pour $\alpha = \frac{\pi}{4}$ et y vaut 1.25, là où les deux autres algorithmes ont le ratio $\sqrt{2}$[1]. Pour cette plage d'angles, l'angle optimal β est donné par la formule explicite : $\beta = \frac{1}{2}\arccos\left(\frac{-2\cos(4\alpha)+\cos(6\alpha)+2}{3-2\cos(4\alpha)}\right)$[2]. Pour notre configuration exacte où $\alpha = \frac{\pi}{4}$, cet angle optimal vaut $-2\arctan\left(3-\sqrt{10}\right)$, soit environ 0.3217 radians[2].

3 Sujet du stage

Le but du stage est de concevoir et analyser expérimentalement un algorithme qui pourrait apprendre des requêtes passés pour anticiper les requêtes à venir.

Le modèle que nous considérons est le suivant. Les points de l'instance du problème ne sont pas générés de manière adverserielle comme c'est supposé dans le cadre d'une analyse du ratio de compétitivité, mais de manière aléatoire. La position de chaque point est tiré de manière indépendant selon une loi normale de moyenne μ et de variance σ^2. Ces paramètres ne sont pas connus par l'algorithme. Cependant il peut apprendre en observant les requêtes passés en utilisant la technique d'inférence Bayesienne.

Le stage qui dure un peu plus que 4 semaines comportera les étapes suivantes.

- Lire des notes de lectures pour comprendre comment apprendre les paramètres μ et σ^2 à partir des observations.
- En parallèle trouver le meilleur algorithme B pour le coût en moyenne qui connaît les paramètres μ, σ^2.
- Combiner le dernier algorithme avec β-HEDGE, pour proposer un algorithme A pour ce modèle.
- Comparer expérimentalement A, B et β-HEDGE.

3.1 Our results

In Fig. 2

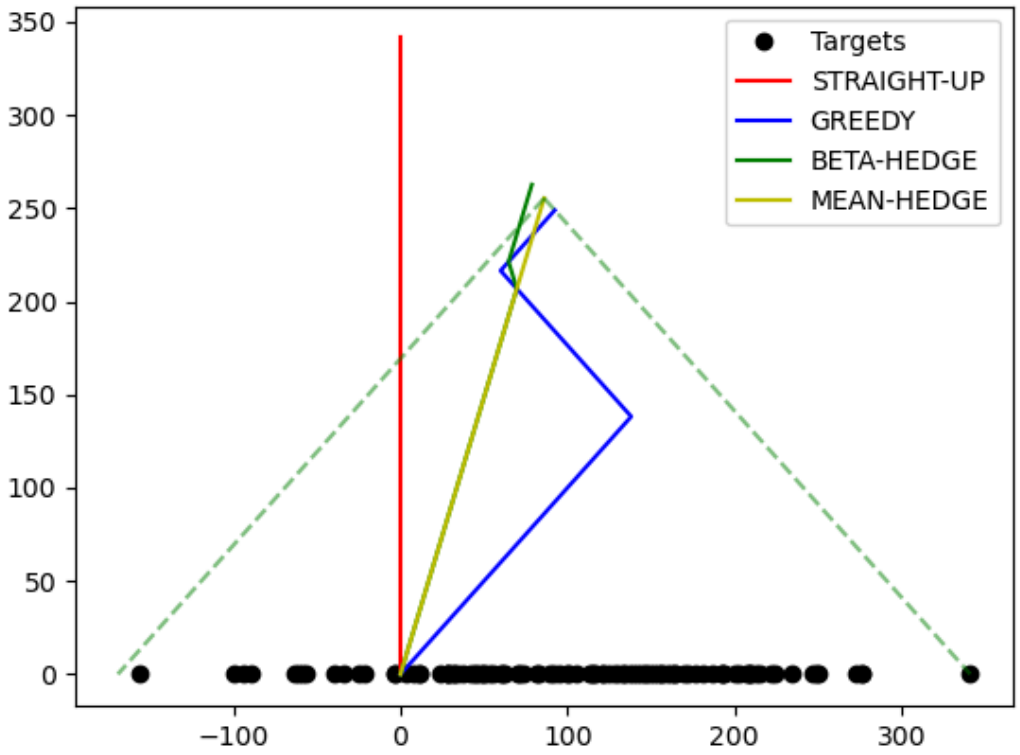


Fig. 2: Trace of different algorithms for 1000 points drawn from $N(100, 100)$

To address this, we propose two learning-augmented algorithms based on the historical mean: Learning-Beta-Up and Learning-Greedy-Up.

Suppose the drone is at position (x, y), and the set of current requests is R. We define the empirical mean $\hat{\mu} = \frac{\sum_{p_i \in R} p_i}{|R|}$. It is well-known that $\hat{\mu}$ is an unbiased estimator of the mean μ, with precision increasing with $|R|$.

Let (x', y') be the baseline target position chosen to serve the new request. The computation of this target depends on the chosen variant:

- For Learning-Beta-Up, (x', y') is calculated using the β-Hedge strategy, meaning its baseline ascent is based on the flight angle β.
- For Learning-Greedy-Up, (x', y') is calculated using the Greedy strategy, meaning its baseline ascent is based on the camera sensor half-angle α.

Once the baseline target (x', y') is determined, both algorithms apply the following movement logic:

- If the current position x and the target x' lie on different sides of the empirical mean $\hat{\mu}$ (i.e., $x < \hat{\mu} < x'$ or $x' < \hat{\mu} < x$), the drone computes the final destination

$(\hat{\mu}, y'')$ required to cover the points, and moves directly to this point in a straight line.
- Otherwise, it moves directly to (x', y').

4 Roadmap

(July 13th, 2026)

For given n, μ, σ, the random instance consists of a sequence of n points, each drawn independently from $N(\mu, \sigma^2)$. We consider 3 different modes to serve these points.

In addition we consider two types of algorithms, both parameterized by some angle β. The first one, called ALG(β), is memoryless and moves at angle β to serve the new request. The second one, called ALG'(β), has an estimation $\hat{\mu}$ of μ, simply from the average of all request points seen so far. It attempts to move as the first algorithm, but once it hits the $x = \hat{\mu}$ vertical line, it continues to move straight up, that is at angle 0. These two moves define the final point to serve the given request, and the algorithm of course moves with a shortcut to this point, i.e. its cost is the Euclidian distance between the previous and the new position.

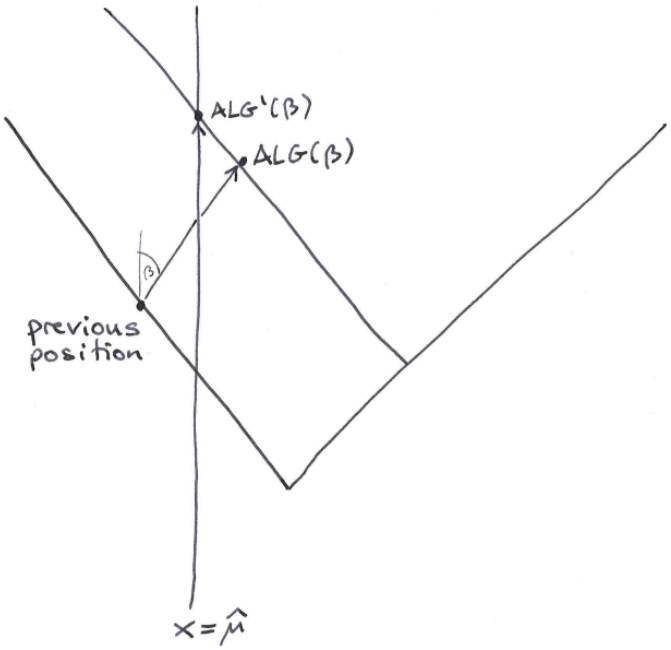


Fig. 3: Illustration of ALG(β) and ALG$'$(β)

4.1 Offline

In the offline setting the algorithm knows the wholte request sequence in advance. The optimal serving depends only the two extremal request points. It is in fact the projection of $(0, 0)$ on the final cone. The formal expression of the optimum is given in Lemma 2 of [2]. By OPT(n, μ, σ) we denote the expected optimal cost.

4.2 Semi-online

The algorithm knows n, μ, σ, but needs to serve each random request, without knowing the request points still to come. However, since the algorithm knows n, it knows how many request points are still to come. This means for example that the last point is optimally served by GREEDY.

So far we don't know what the best semi-online algorithm is for given n, μ, σ. Therefore we will conduct a few experiments, for several values of n, μ, σ with $n \in \{2, 4, 8, ..., 1024\}$, $\mu \in \{0, 10, 20, 30, ..., 200\}$ and $\sigma \in \{10, 20, 30, ..., 200\}$. For each $\beta \in \left\{0, \frac{1}{16}\alpha, \frac{2}{16}\alpha, \frac{3}{16}\alpha, ..., \alpha\right\}$, we compute the average cost of $ALG'(\beta)$ over a sufficiently large number of random instances. Eventually we obtain a collection of data vectors of the form $(n, \mu, \sigma, \beta, c)$, where c is the average cost of $ALG'(\beta)$ normalized by $OPT(n, \mu, \sigma)$.

We then feed this collected data to a learning algorithm to train an estimator B, such that $B(n, \mu, \sigma)$ predicts the angle β yielding the best competitive ratio. The supervised learning pipeline is defined as follows:

1. **Data Generation & Labeling:** Through grid search simulations, we identify the optimal angle β_{best} that minimizes the normalized cost c for each parameter combination. This β_{best} serves as our target continuous label.
2. **Coefficient of Variation:** To make the model scale-invariant and prevent gradient instability during training, we input not only the empirical parameters but also the Coefficient of Variation, defined as $CV = \frac{\hat{\sigma}}{\hat{\mu}+1}$.
3. **Model Architecture:** We employ a Multilayer Perceptron (MLP) with a bottleneck architecture of $4 \to 64 \to 128 \to 64 \to 1$ layers. The expansion to 128 neurons captures non-linear geometric relations, while the subsequent compression to 64 acts as a filter against spatial noise. The network uses ReLU activations and outputs a continuous value for β.
4. **Training & Inference:** The network is trained as a regression task to minimize the Mean Squared Error (MSE) between the predicted β and the target β_{best}, optimized via Adam. In the online setting, the model takes the dynamic empirical estimates $(n, \hat{\mu}, \hat{\sigma}, CV)$ as input and outputs the optimal angle dynamically.

4.3 Online

The algorithm does not know n, μ, or σ. We propose the following algorithm: the very first request will be served by GREEDY. From that moment on, n will be the smallest integer power of two such that n is at least the number of request points seen so far (including the current request). Additionally, $\hat{\mu}$ is the empirical average of the request points, and $\hat{\sigma}^2$ is the sample variance of the points seen so far. The algorithm will follow ALG$'(n, \hat{\mu}, \hat{\sigma})$.

5 Experimental Evaluation

To evaluate the performance of our proposed approaches, we developed a Python simulation environment. The primary goal is to compare the competitive ratios of our learning-augmented algorithms against classic online strategies.

5.1 Setting Up

We evaluate the algorithms on randomly generated instances. For each instance, a sequence of n points is drawn independently from a normal distribution $N(\mu, \sigma^2)$. To thoroughly explore the parameter space, we generated sequences with the following parameters:

- Number of points: $n \in \{2, 4, 8, ..., 1024\}$
- Mean of the distribution: $\mu \in \{0, 10, 20, ..., 200\}$
- Standard deviation: $\sigma \in \{10, 20, 30, ..., 200\}$

5.2 Evaluated Algorithms

Let $p_{\min}$ and $p_{\max}$ denote the leftmost and rightmost points to be covered by the drone at any given request. We remind aswell that alpha is fixed as $\alpha = \frac{\pi}{4}$

- STRAIGHT-UP: The drone moves strictly vertically until its vision cone covers both $p_{\min}$ and $p_{\max}$. Meaning it moves to $(0, y)$ with $y = \max\{p_{\min}, p_{\max}\}$.
- GREEDY: The drone computes the spatial midpoint between $p_{\min}$ and $p_{\max}$ and moves directly towards this center along the shortest valid trajectory. Meaning it moves to $\left(\frac{p_{\min}+p_{\max}}{2}, \frac{p_{\max}-p_{\min}}{2}\right)$.
- BETA-HEDGE: Proposed by Dobrev et al. [1], this strategy acts as a trade-off between the vertical and greedy movements. The drone travels at a fixed flight angle β with respect to the vertical axis to reach the closest valid point on the feasibility cone [2]. Following the theoretical bounds for a sensor half-angle $\alpha = \frac{\pi}{4}$, we fix $\beta = \frac{\pi}{9.666}$ to minimize the worst-case competitive ratio [2].
- LEARNING-BETA-UP: A semi-online approach tracking the statistical mean of all previously seen requests. It computes the target x-coordinate using the BETA-HEDGE logic. If the historical mean lies between the drone's current position and this target, the drone moves to the mean's x-coordinate and proceeds straight up. Otherwise, it executes the standard BETA-HEDGE movement.
- LEARNING-GREEDY-UP: A learning variant of the greedy strategy. If the historical mean lies between the drone's current position and the midpoint of $p_{\min}$ and $p_{\max}$, the drone moves horizontally to the mean and goes straight up. Otherwise, it defaults to the GREEDY algorithm.
- MWU (Multiplicative Weights Update): This algorithm discretizes the possible flight angles into a finite set, maintaining a probability weight for each. For every request, it stochastically selects an angle based on the weight distribution. Post-movement, it simulates the travel cost for all angles and penalizes the weights of underperforming ones using a learning rate η.

5.3 Results and Discussion

Simulations were conducted over $k = 100$ random instances, with each sequence consisting of $n = 500$ target requests. Performance is measured by the competitive ratio (the algorithm's cost divided by the optimal offline cost). We analyzed the impact of the distribution's mean (μ) and standard deviation (σ) on the total distance flown.

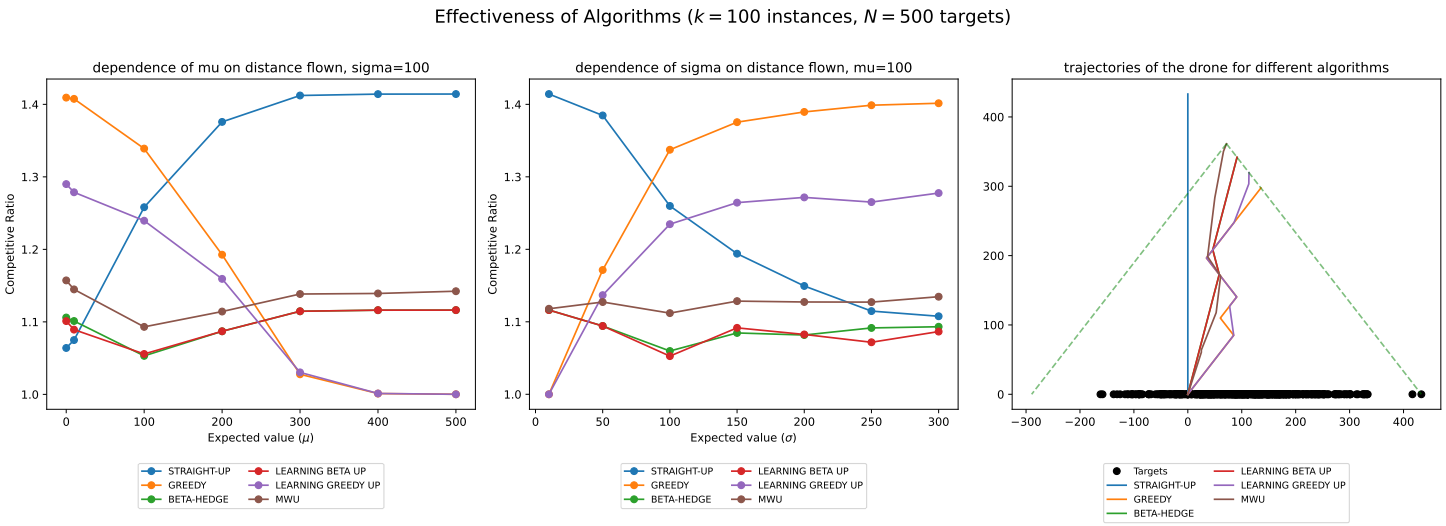


Fig. 4: Effectiveness of Algorithms and Trajectory Analysis

5.3.1 Impact of the Mean (μ)

The first experiment fixes the standard deviation at $\sigma = 100$ and varies μ from 0 to 650. **The STRAIGHT-UP algorithm performs well when μ is close to the origin but degrades as the cluster of points shifts further away due to the lack of horizontal movement.** In contrast, the standard GREEDY algorithm struggles with origin-centered points but approaches the optimal cost for large μ, where all requests fall on one side and the trajectory becomes a direct diagonal. The learning-augmented algorithms show better stability. LEARNING-GREEDY-UP outperforms its memoryless counterpart across the tested spectrum. Both BETA-HEDGE and LEARNING-BETA-UP are the most efficient, maintaining a flat competitive ratio between 1.05 and 1.1 regardless of the distribution shift.

5.3.2 Impact of the Variance (σ)

The second experiment fixes the mean at $\mu = 100$ and varies σ from 10 to 300 to test resilience to data spread. **As σ increases, the wider spread of targets causes the standard GREEDY algorithm to overreact to distant outliers, resulting in inefficient zigzag trajectories and a higher competitive ratio.** The LEARNING-GREEDY-UP algorithm smooths out these erratic movements by relying on the historical mean, dominating the standard GREEDY approach as variance grows. LEARNING-BETA-UP and BETA-HEDGE remain largely unaffected by variance, yielding the lowest overall competitive ratios. The stochastic MWU algorithm also shows stability, maintaining a ratio near 1.15.

5.3.3 Trajectory Analysis

The visual trace of the trajectories aligns with the quantitative results. While the GREEDY algorithm exhibits sharp directional changes, the learning-augmented algorithms (such as LEARNING-GREEDY-UP and MWU) estimate the distribution parameters early. By utilizing the statistical mean, they optimize their flight paths to reach the center of mass before expanding their vision cones, reducing unnecessary horizontal travel.

6 Neural Network Integration

To to advance our research even further, we train a Multilayer Perceptron (MLP) to dynamically predict the optimal flight angle β at each step.

6.1 Features and Architecture

The model processes four input features: the current empirical estimates $\hat{n}$, $\hat{\mu}$, $\hat{\sigma}$, and a specialized Coefficient of Variation (CV). The CV is mathematically defined as $CV = \frac{\hat{\sigma}}{\hat{\mu}+1}$. By normalizing the target spread relative to the distance, this metric provides a scale-invariant measure of cluster density, thereby preventing gradient instability during backpropagation.

We use MLP architecture with layer dimensions $4 \to 64 \to 128 \to 64 \to 1$. The initial expansion to 128 neurons increases the model's expressive capacity to capture highly non-linear geometric relations, while the subsequent compression to 64 serves as a regularization mechanism, acting as a spatial noise filter against overfitting. The network employs ReLU activations and outputs a continuous scalar for β.

While the model functions as a continuous regressor, its operational predictions naturally exhibit a tendency toward the 15 discrete β values originally sampled during the grid-search data generation phase. The Mean Squared Error (MSE) loss heavily penalizes deviations from these specific targets during training. Consequently, the model retains mathematical flexibility but effectively acts as a high-precision interpolator, mathematically anchored to the primary attractor states defined by the training set.

6.2 Training Process

The objective of the supervised learning phase is to map the latent geometric and statistical correlations between the ground target distribution and the optimal flight trajectory. During the training loop, the algorithm generates a predicted flight angle and directly compares it against the true optimal angle (β_{best}) provided by the dataset. This comparison is quantified using the Mean Squared Error (MSE) loss function. The computed mathematical difference (error) is then propagated backward through the network. Based on this exact error margin, the Adam optimizer (a PyTorch-integrated optimizer) redistributes and updates the network's internal weights, adjusting the mathematical influence of each node to minimize the loss in future predictions. Over hundreds of training loops, the model converges, successfully internalizing the mathematical mapping for optimal flight.

6.3 Strategy

Applying a model trained on true distribution parameters to an online setting introduces a semantic gap: empirical estimates $(\hat{\mu}, \hat{\sigma})$ are highly noisy and statistically insignificant for small sample sizes.

To maintain a strictly online algorithm without degrading early performance, we use an Explore-then-Exploit framework:

1. **Explore, $n < 20$:** When the empirical data is statistically insignificant, the algorithm bypasses the neural network. The drone defaults to a robust baseline

movement (STRAIGHT-UP, $\beta = 0$). This strategy safely expands the vision cone without risking erroneous horizontal maneuvers driven by noisy initial data.

2. **Exploit, $n \geq 20$:** Once the sample size reaches a reliable threshold, the algorithm transitions into the machine learning-driven regime. The control system feeds the continuously updated empirical variables $(\hat{n}, \hat{\mu}, \hat{\sigma}, CV)$ directly into the pre-trained MLP. Leveraging its calibrated weight matrices, the network dynamically computes the optimal continuous angle β for each subsequent request in real-time.

6.4 Statistical Analysis of the ML Model

The experimental results evaluate the LEARNING ML algorithm against the baseline heuristics over $k = 100$ instances with $N = 500$ targets. The analysis focuses on the competitive ratio relative to the distribution parameters μ and σ.

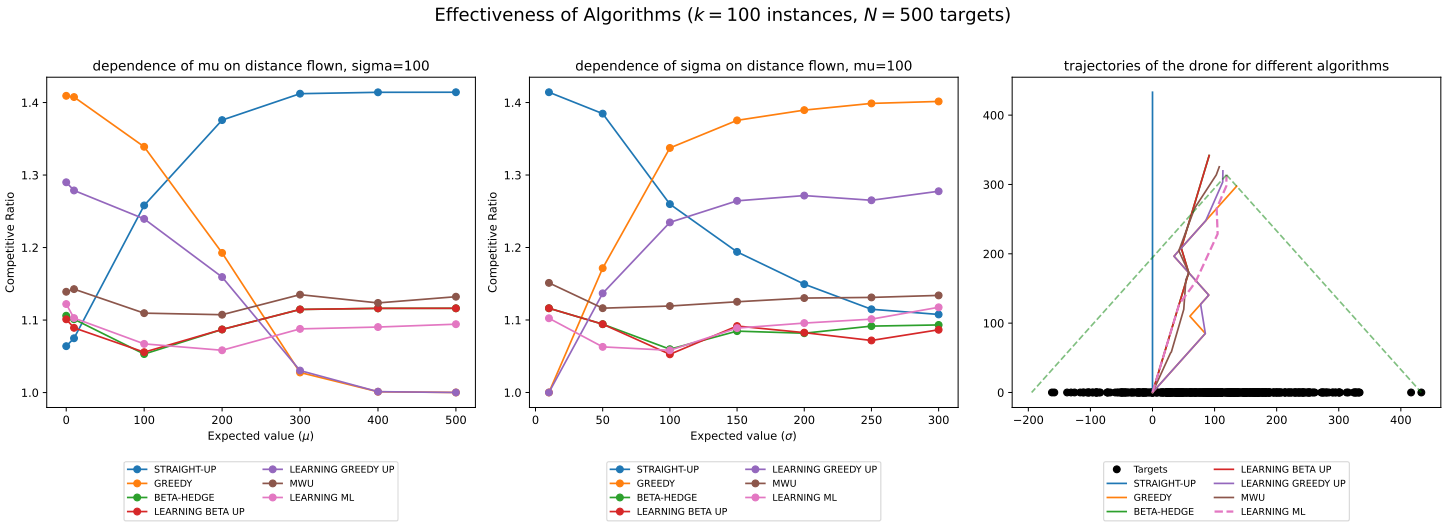


Fig. 5: Effectiveness of Algorithms and Trajectory Analysis

Impact of the Mean (μ): With the standard deviation fixed at $\sigma = 100$, the LEARNING ML algorithm demonstrates strict stability across all tested expected values ($\mu \in [0, 500]$). It maintains a flat competitive ratio between 1.07 and 1.12. This contrasts sharply with the STRAIGHT-UP algorithm, which degrades as the cluster of points shifts further away due to the lack of horizontal movement. It also avoids the high initial costs of the GREEDY algorithm near the origin. The ML model successfully mirrors the optimal, flat scaling of BETA-HEDGE and LEARNING BETA UP.

Impact of the Variance (σ): With the mean fixed at $\mu = 100$, LEARNING ML shows high resilience to data spread across $\sigma \in [10, 300]$. As variance increases, the wider spread of targets causes the standard GREEDY algorithm to overreact to distant outliers, resulting in inefficient zigzag trajectories and a higher competitive ratio. In contrast, the neural network model keeps the competitive ratio tightly bounded between 1.06 and 1.10.

Trajectory Analysis: The visual trace confirms the Explore-then-Exploit mechanics. The LEARNING ML trajectory avoids the severe directional zigzags characteristic

of the standard GREEDY approach. By leveraging the dynamic coefficient of variation, the neural network continuously refines its flight angle. After the initial vertical warm-up phase, it adjusts the trajectory smoothly, resulting in an optimized path that converges directly on the target center of mass.

Bibliographie

[1] S. Dobrev *et al.*, « Online Drone Coverage of Targets on a Line », *arXiv preprint arXiv:2604.02491*, 2026.

[2] S. Dobrev *et al.*, « Online Drone Coverage of Targets on a Line », in *Combinatorial Algorithms*, F. Foucaud et A. Parreau, Éd., Cham: Springer Nature Switzerland, 2026, p. 266-280.